

Android

Baze podataka
Threading, SQLite, Room

Sadržaj predavanja

.Multithreading

- Što raditi, a što izbjegavati
- Tehnike multithredinga na Androidu

.SQLite

- Kreiranje baze podataka
- Upiti
- Room

Osnovni pojmovi

Dretva

- Izvršavaju se unutar procesa (kod većine OS-a)
- Dijelev resurse (Memorija tj. adresni prostor)
 - Istoj varijabli mogu pristupiti različite dretve
- Imaju zaseban povratni stog (engl. stack)
- Na sustavu s jednim procesorom događa se *time-division multiplexing*
 - Prebacivanje s jedne na drugu dretvu
 - Korisnik ima dojam da se operacije izvode simultano
- Na višeprocorskim sustavima dretve se izvode simultano ovisno o broju procesora / jezgara

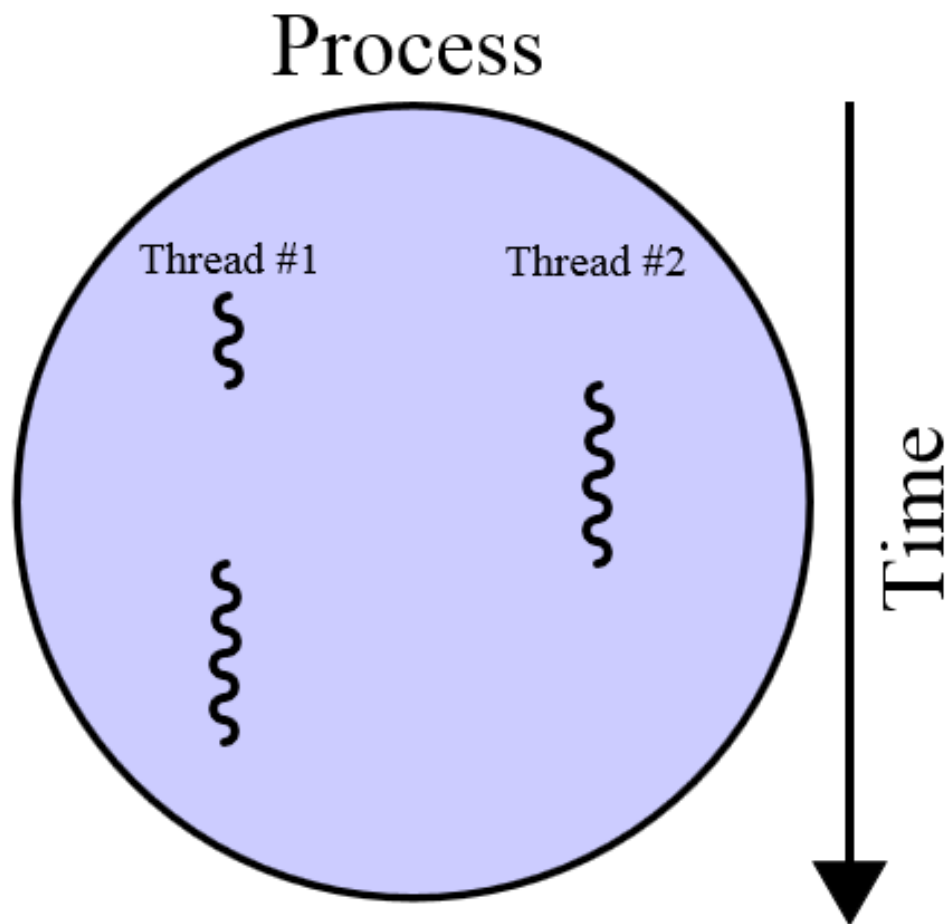
Osnovni pojmovi

Proces

- Instanca programa koji se izvodi
- Jedan program može imati više instanci
- OS brine da procesi imaju odvojene resurse (nemoguće je pristupiti varijabli drugog procesa iz prvog procesa)

Osnovni pojmovi

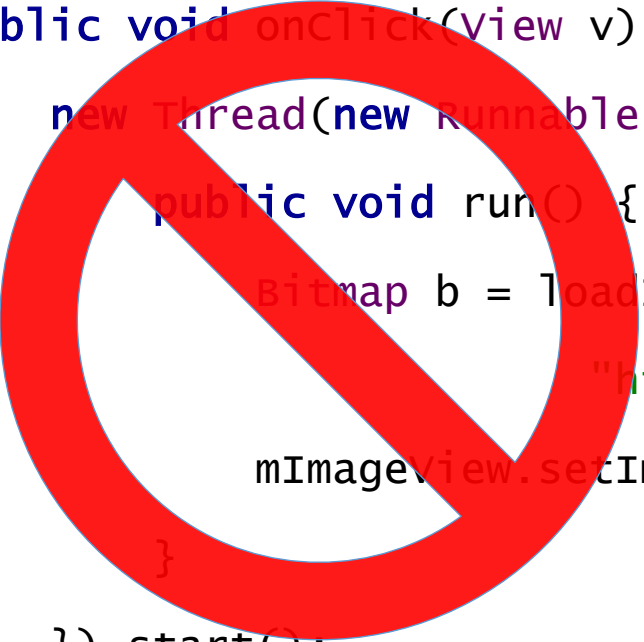
Proces vs. Dretva



Višedretvenost na Androidu

Višedretvenost

Moguće je koristiti Klasičan Java kod kako bi osvježili UI iz druge dretve:



```
public void onClick(View v) {  
    new Thread(new Runnable() {  
        public void run() {  
            Bitmap b = loadImageFromNetwork(  
                "http://example.com/image.png");  
            mImageView.setImageBitmap(b);  
        }  
    }).start();  
}
```

NE MODIFICIRAJTE UI IZ DRUGE DRETVE!!!

Višedretvenost

Iz druge dretve UI je potrebno osvježavati na sljedeći način:

```
public void onClick(View v) {  
    new Thread(new Runnable() {  
        public void run() {  
            final Bitmap bitmap = loadImageFromNetwork(  
                "http://example.com/image.png");  
            mImageView.post(new Runnable() {  
                public void run() {  
                    mImageView.setImageBitmap(bitmap);  
                }  
            });  
        }  
    }).start();  
}
```


Višedretvenost

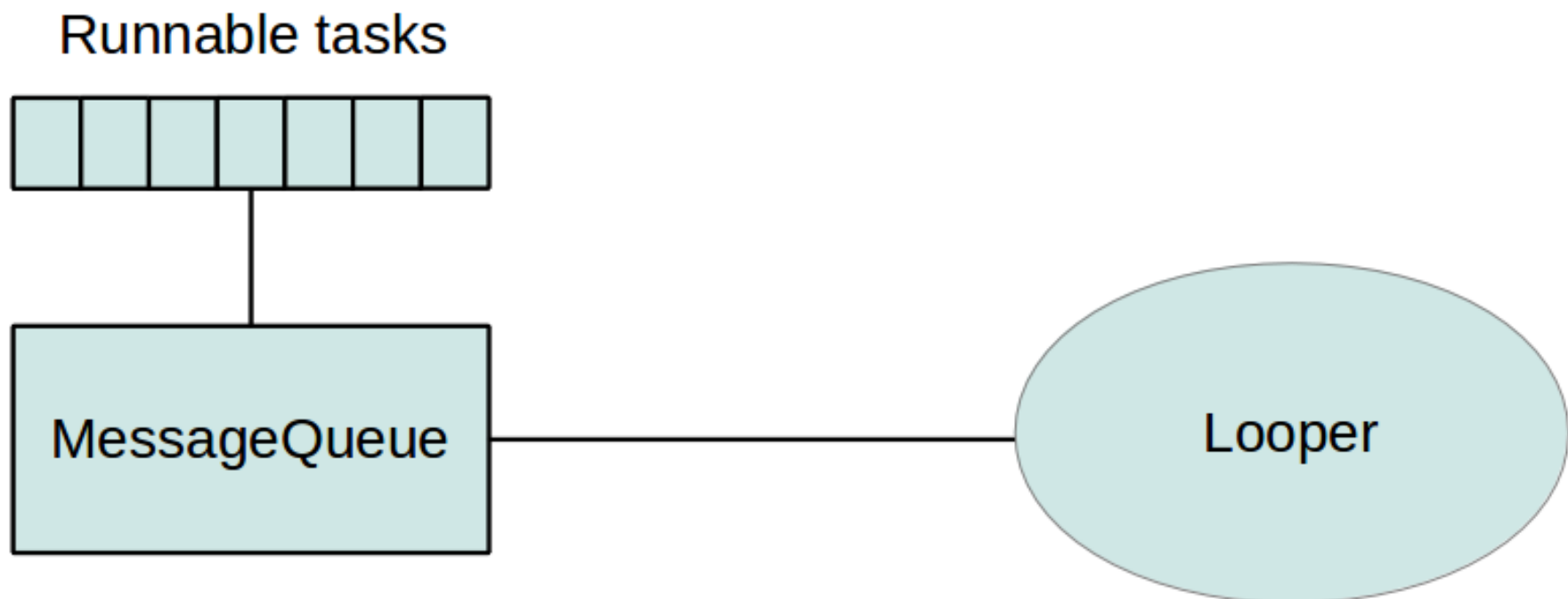
Dva pravila

- NIKADA SE NE SMIJE BLOKIRATI UI DRETVA
 - Android nakon 5 sekundi gasi aplikaciju
- NIKADA SE NE SMIJE PRISTUPITI UI DRETVI IZ DRUGE DRETVE DIREKTNO
 - Može se dogoditi neregularno i nasumično ponašanje koje je jako teško pronaći i ukloniti

A zašto?

Zbog *looper*a

- Android vrti jednu glavnu UI dretvu
- Unutar nje postoji Looper koji izvršava zadatke iz message queue-a



Pojašnjenje

Zašto baš dretve?

- Ako se program piše u istoj dretvi, naredbe se izvršavaju jedna nakon druge
- Ako je radnja dulja, aplikacija se blokira dok god se radnja ne izvrši (prošli slajd)
- Kako bi korisnikov doživljaj aplikacije bio što bolji, blokiranje korisničkog sučelja nije dobrodošlo

Što koristiti na Androidu?

- Android posjeduje iste klase za dretve kao i Java +
- Handler
- Headless fragmenti (fragmenti bez GUI-a)
- Loader
- Servisi
- Cache
- LiveData...

Klasične dretve

(Kao u Javi)

- Rade identično kao u javi
- Moguće je koristiti Thread, ThreadPools i Executor
- Nedostaci:
 - Sinkronizacija s glavnom UI dretvom
 - Nema automatskog zaustavljanja dretve
 - Nema automatskog thread poolinga
 - Nema automatskog praćenja promjena stanja Androida

Klasične dretve



Kada će se raspasti

Korišćenje post metode

Handler klasa

(Ubacivač poruka u red poruka)

- Koristi se kao pomoćna klasa za ubacivanje poruka u message queue iz druge dretve
- Dretva i dalje postoji, a Handler samo pomaže oko sinkronizacije s GUI-jem
- Moguće ju je koristiti na više načina
 - Preko Bundlea dodavati poruke
 - Pomoću post metode (slično kao kod obične dretve)

Handler



Načini korišćenja Handler
klase

SQLite

- SQLite je relacijska baza podataka
- Posebna je po tome što je idealna za embedded sustave (ne vrti zaseban proces)
- Ponaša se kao library (Zapravo i je library)
- Napisan u C-u, wrappan u Javu
- Radi pomoću SQL jezika

SQLite

Na Androidu

- Svaka aplikacija ima svoju bazu
- Baze aplikacija se nalaze u data/data direktoriju
- Nevjerojatno, ali moguće im je pristupiti preko ADB-a (na emulatoru)
- Također ih je moguće kopirati i onda raditi s njima na računalu

SQLite

Hacking the SQLite

•Vezanje na uređaj:

- adb -e shell

•Prelazak u direktorij i kreiranje novog:

- cd /data/data
- cd hr.android.primjeri/
- cd databases

•Igranje s bazom:

- sqlite3 ./naziv_baze

•Moguće je kreirati i vlastitu bazu

Baze iz Android aplikacije

.Zato što su baze wrappane u java kod, to znači da im je moguće pristupiti bez nekih većih poteškoća

.SQLiteOpenHelper

- Sadrži gotove metode za rad s bazom
- Nju je potrebno naslijediti za lakši rad

.Cursor

- Klasa za pristup resultsetu (Kod select naredbe npr.)

.SQLiteDatabase

- Klasa koja služi kao interface između programskog koda i baze
- Uključuje funkcije za SQL operacije poput selecta, inserta i deletea

.Model klase

Baze iz Android aplikacije

- Za insert, update i delete postoje metode koje generiraju upit
- Moguće je izvršavati praktički bilo koji SQL upit
- Idealno bi bilo da se rad s bazama izvodi u zasebnoj dretvi kako se aplikacija ne bi „smrznula”
- Potrebno je naslijediti SQLiteOpenHelper u kojoj se nalaze metode za pristup bazi i rad s bazom
- Postoji i odličan framework za rad s bazama - Room

Baze iz Android aplikacije



Izrada nove baze

Kreiranje upita

ORM

Object-relational mapper

- Tehnika pretvaranja podataka između nekompatibilnih tipova u objektno orijentiranim jezicima
- Kreira se virtualna objektna baza podataka
- Npr:

```
String sql = "SELECT ... FROM persons WHERE id = 10";  
DbCommand cmd = new DbCommand(connection, sql);  
Result res = cmd.Execute();  
String name = res[0]["FIRST_NAME"];
```

Prevedeno u OOP:

```
Person p = repository.GetPerson(10);  
String name = p.getFirstName();
```

Room

Object-relational mapper

- Room biblioteka omogućava jednostavniji pristup podacima pohranjenim u SQLite bazi podataka
- Dodajte sloj apstrakcije
- Prednosti:
 - Verifikacija upita tijekom kompajliranja
 - Anotacije minimiziraju ponavljajući kod i smanjuju mogućnost pogrešaka
 - Jednostavnost migracija

Room

Arhitektura

.Sastoji se od 3 glavne komponente:

- Klasa baze podataka
- Klase koje predstavljaju tablice
- DAO (Data Access Objects) klase koje sadrže metode za CRUD operacije

Room

Postavljanje

.Potrebno je dodati dependencies:

```
def room_version = "2.7.1"

implementation "androidx.room:room-runtime:$room_version"
ksp "androidx.room:room-compiler:$room_version"
```

.Definirati entitete:

```
@Entity
```

```
data class User(
    @PrimaryKey val uid: Int,
    @ColumnInfo(name = "first_name") val firstName: String?,
    @ColumnInfo(name = "last_name") val lastName: String?
)
```

Room

Postavljanje

.Kreirati DAO:

@Dao

```
interface UserDao {  
    @Query("SELECT * FROM user")  
    fun getAll(): List<User>  
    @Insert  
    fun insertAll(vararg users: User)  
    @Delete  
    fun delete(user: User)  
}
```

Room

Postavljanje

.Definirati bazu podataka:

```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

.Kreiranje instance baze podataka:

```
val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java, "database-name"
).build()
```

Room

Postavljanje

.Rad sa podacima:

```
val userDao = db.userDao()
```

```
val users: List<User> = userDao.getAll()
```

Room



Room

finish();



•pitanja?